CSim

Infinite Canvas Implementation Guide

A detailed, decision-complete engineering plan for replacing the finite toroidal grid with a sparse, signed-coordinate, chunked world while preserving CSim's rules, editor, RGB fade, token renderer, and headless testability.

Design choice	Locked result
Domain	Sparse hash map of dense 16 x 16 byte chunks
Neighbors	Temporary 18 x 18 halo; no persistent copied borders
Topology	Infinite plane semantics; no toroidal wrap
Rendering	One visible-region RGB texture and one quad
Visuals	Visible-area fade; newly exposed cells snap
Persistence	Read legacy dense files; write versioned sparse files
Compute	Deterministic serial v1; benchmark before parallel work

How to use this guide
Implement one phase at a time. Keep the headless suite green after each phase, preserve the existing dirty worktree, and do not begin rendering or file migration until the sparse domain passes cross-chunk equivalence tests.

Prepared 2026-08-02

Contents

Repository baseline
The live Release CSimTests target built and passed on 2026-08-02 before this guide was generated. Re-run that baseline locally before making changes because the checkout already contains unrelated staged deletions and modifications.

1. Outcome and boundaries

The finished simulator has no allocated rectangular canvas and no edge that wraps. A finite amount of non-background state may exist anywhere on a conceptually unbounded integer grid. Editing, simulation, rendering, clearing, rule switching, save/load, camera reset, and startup all use that same sparse world.

Success criteria

- Painting at negative or distant coordinates creates only the touched chunks.
- A pattern crossing a 16-cell boundary behaves exactly like the same pattern away from a boundary.
- Generation N+1 is computed only from generation N; iteration order never changes results.
- No cell on one side of the occupied region can influence a distant cell through wrapping.
- Empty chunks are reclaimed immediately from domain storage.
- The draw-command count remains constant as the world accumulates chunks.
- All currently active rulesets remain supported, including multi-state Brian's Brain and Wireworld.
- New-format files preserve distant coordinates, the active ruleset, and the saved camera view.
- Legacy dense files load at positive coordinates beginning at (0,0).

Explicit non-goals

- No ECS, per-cell objects, generalized scene graph, or per-chunk GPU object hierarchy.
- No persistent 18 x 18 chunks; copied halo cells are scratch data only.
- No CPU threading, SYCL backend, compute abstraction, or second finite-canvas mode in v1.
- No support for rules whose empty background spontaneously becomes non-empty with zero active neighbors.
- No zoom-level aggregation or distant-world level of detail beyond the current practical zoom range.

> **Core correction to the naive model**
> Do not create a list of live cell objects. CSim has multi-state rules, so the stored concept is a non-background byte. Dense 16 x 16 chunks preserve cache locality and make cross-boundary reads predictable without allocating one object per cell.

2. Current code and pressure points

Today Canvas owns the dense lifeCanvas, the targetRgb and displayRgb fade arrays, dirty rectangles, the RGB upload buffer, and enrolled GPU resources. RuleSet scans the entire width x height array and wraps neighbor indices at every edge. CellGameModule converts mouse coordinates into bounded integer cells and save/load writes a dense rectangle.

Current responsibility	Where it lives	Why it must change
Cell bytes	Game/Canvas	A single width x height allocation cannot be infinite.
Generation sweep	Rulesets/RuleSet	The full-grid toroidal scan makes size, topology, and rule transition inseparable.
Fade buffers	Game/Canvas	Global RGB float arrays scale with canvas area rather than visible work.
GPU texture	Game/Canvas	One texture currently represents the entire finite domain.
Camera center	Rendering/Camera	CanvasX/CanvasY define the initial center and reset location.
Editor bounds	Game/CellGameModule	Paint strokes explicitly reject coordinates outside width and height.
Save format	Game/CellGameModule	The file encodes fixed width, fixed height, and width x height bytes.

Boundaries to preserve

- Rules remain independent of input and raw OpenGL.
- Production drawables continue to emit renderer tokens through IBackend.
- CellMain continues to compose modules; Illumo does not construct Game types.
- MockBackend remains the proof path for headless rendering tests.
- The default build still compiles and executes CSimTests.

Baseline commands

```
git status --short
cmake -S CSim -B build
cmake --build build --config Release --target CSimTests
ctest --test-dir build -C Release -R "^CSimTests$" --output-on-failure
```

Worktree safety
Do not clean, reset, restore, or normalize the checkout before starting. Record the existing status and restrict each phase to the files named by that phase.

3. Target architecture

The scale requirement is the concrete reason to supersede the current decision that Canvas should remain a combined domain and view type.

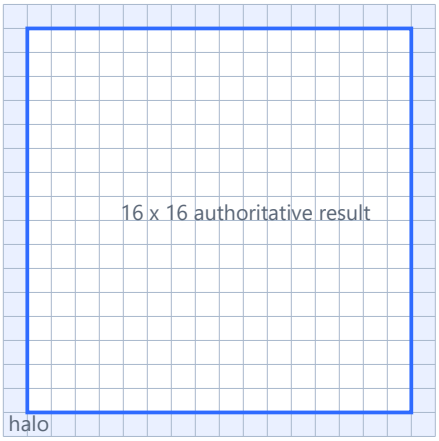
Layer	New responsibility	Must not own
SparseCellGrid	Authoritative cell state, chunks, coordinate mapping, generation swap, revision	Camera, palette, texture, renderer
RuleSet	Pure cell transition and palette mapping	Grid, CanvasView, input, GPU resources
CanvasView	Visible rectangle, fade buffers, staging texture, draw tokens	Authoritative world state
CellContext	Own grid, active rule, and view; coordinate their lifetimes	Platform dialogs
CellGameModule	Editor, tick scheduling, camera controls, file actions	Raw cell buffers or OpenGL
CellWorldIO	Versioned serialization and legacy parsing	Dialogs, renderer, live world mutation during parse

Ownership order

- CellContext constructs SparseCellGrid first, a canvas-independent RuleSet second, and CanvasView last with non-owning pointers to grid, window, camera, and renderer.
- CellContext destroys CanvasView before RuleSet and SparseCellGrid.
- CellGameModule registers console file handlers during Start and clears them before destroying CellContext during Exit.
- No new pointer is added to IllumoContext; its existing commandLine pointer is sufficient.

4. Sparse chunk data model

Temporary 18 x 18 read tile



Nine old-map lookups

NW N NE

W C E

SW S SE

Copy edges and corners into the halo.

Evaluate 256 central cells contiguously.

Never synchronize persistent borders.

Only the next 16 x 16 result is retained.

Authoritative structures

```
using CellCoordinate = std::int64_t;

struct CellAddress {
    CellCoordinate x;
    CellCoordinate y;
};

struct ChunkAddress {
    std::int64_t x;
    std::int64_t y;
    bool operator==(const ChunkAddress& other) const;
};

struct CellChunk {
    static const int Size = 16;
    static const int CellCount = Size * Size;
    std::array<unsigned char, CellCount> cells;
    std::uint16_t nonBackgroundCount;
};

class SparseCellGrid {
public:
    static const unsigned char BackgroundState = 1;
    static const unsigned char CountedNeighborState = 0;

    unsigned char getCell(const CellAddress& address) const;
    bool setCell(const CellAddress& address, unsigned char state);
    void clear();
    bool advance(const RuleSet& ruleSet);
    std::uint64_t getRevision() const;
    std::size_t getAllocatedChunkCount() const;
};
```

Map and chunk invariants

- The unordered_map contains no chunk whose nonBackgroundCount is zero.
- Every absent chunk and absent cell reads as BackgroundState (1).
- nonBackgroundCount exactly equals the number of bytes not equal to 1.
- setCell increments the revision only when the stored logical value changes.

- Writing state 1 into an absent cell is a successful no-op and allocates nothing.
- Writing the last non-background cell back to state 1 erases its chunk.
- The map key hash combines both signed 64-bit coordinates without narrowing them.

Why not a live-cell set

A set of live coordinates works only for binary rules with one interesting state. Wireworld needs head, tail, and conductor; Brian's Brain needs alive and dying. A byte chunk represents all current rules uniformly and evaluates 256 neighboring cells with contiguous memory.

Representation	Strength	Failure for this project
Cell objects	Simple mental model	Allocation, pointer chasing, and no natural multi-state neighborhood tile
Set of live coordinates	Very sparse binary worlds	Loses conductor, tail, and dying states or requires several sets
Dense global grid	Fast scan	Memory and work scale with coordinate span
Dense sparse chunks	Multi-state, cache-local, finite allocation	Requires careful signed coordinate and candidate handling

5. Coordinate math and virtual distance

C++ integer division truncates toward zero, which is wrong for negative chunk coordinates. Centralize floor division and modulo; never duplicate the mapping formula in editor, view, serializer, or tests.

```
static std::int64_t floorDivide16(std::int64_t value)
{
    std::int64_t quotient = value / 16;
    std::int64_t remainder = value % 16;
    if (remainder < 0)
    {
        quotient -= 1;
    }
    return quotient;
}

static int floorModulo16(std::int64_t value)
{
    std::int64_t remainder = value % 16;
    if (remainder < 0)
    {
        remainder += 16;
    }
    return static_cast<int>(remainder);
}
```

World cell	Chunk	Local cell
0	0	0
15	0	15
16	1	0
-1	-1	15
-16	-1	0
-17	-2	15

Camera representation

Do not store an arbitrarily distant camera as one float or one absolute double pixel coordinate. Store an anchor cell and normalized sub-cell pixel offsets. GPU positions are then calculated relative to that nearby anchor and remain small.

```
struct WorldViewPosition {
    std::int64_t anchorCellX;
    std::int64_t anchorCellY;
    double offsetPixelsX;   // normalized around one 16-pixel cell
    double offsetPixelsY;
};

class Camera {
public:
    CellAddress screenToCell(const glm::dvec2& screen) const;
    void panByScreenPixels(const glm::dvec2& delta);
    void zoomAtScreen(float factor, const glm::dvec2& screen);
    CellRect getVisibleCellRect(int paddingCells) const;
    glm::mat4 getRelativeMvp(const CellAddress& renderOrigin) const;
};
```

- Normalize offsets after every pan, zoom, set, load, and smoothing update.
- Reject a pan or edit that would overflow signed 64-bit cell coordinates; log once instead of wrapping.
- Use screen-space mouse deltas for panning rather than subtracting two huge world-space floats.
- Make zoomAtScreen keep the cell and fractional position beneath the cursor fixed.
- Compute CanvasView vertices relative to the camera anchor before converting to float.

6. Generation algorithm

Step 1: determine candidate chunks

A radius-one Moore rule can affect only the current occupied chunk or one of its eight neighbors during a single generation. Insert those nine keys for every allocated source chunk into a temporary candidate set. Deduplication is mandatory.

Step 2: assemble a read-only halo

For each candidate, look up the 3 x 3 neighboring source chunks once. Fill a reusable 18 x 18 byte array: the central 16 x 16 values, four one-cell edges, and four corners. Missing source chunks contribute state 1.

Step 3: evaluate and retain

```
bool SparseCellGrid::advance(const RuleSet& ruleSet)
{
    validateSparseCompatibility(ruleSet);
    CandidateSet candidates = buildCandidates(chunks);
    ChunkMap nextChunks;
    nextChunks.reserve(chunks.size());
    bool anyChange = false;

    for (each candidate in candidates)
    {
        fillHaloFromOldMap(candidate, halo);
        CellChunk next = makeBackgroundChunk();

        for (int localY = 0; localY < 16; ++localY)
        {
            for (int localX = 0; localX < 16; ++localX)
            {
                unsigned char oldState = halo[(localY + 1) * 18 + localX + 1];
                unsigned char neighbors = countStateZero(halo, localX + 1, localY + 1);
                unsigned char nextState = ruleSet.evaluateNext(oldState, neighbors);
                writeAndCount(next, localX, localY, nextState);
                anyChange = anyChange || (nextState != oldState);
            }
        }

        if (next.nonBackgroundCount != 0)
        {
            nextChunks.emplace(candidate, next);
        }
    }

    if (mapsDifferByPresenceOrValue(chunks, nextChunks))
    {
        chunks.swap(nextChunks);
        revision += 1;
        return true;
    }
    return false;
}
```

Important correctness rule
Read only from the old chunk map for the entire generation. Never insert next-state chunks into the source map and never reuse a persistent halo. The final map swap is the simultaneous-update boundary.

Allocation failure

Build candidates and nextChunks as temporaries. Catch allocation failure at the generation boundary, discard the incomplete temporaries, leave the old map and revision unchanged, log the failure, and pause or skip that tick. Never publish a partial generation.

Complexity

- Storage is O(A), where A is the number of allocated non-background chunks.
- One generation evaluates at most 9A candidate keys before deduplication, with 256 central cells per candidate.
- Coordinate distance between chunks does not affect simulation cost.
- A large static Wireworld conductor still remains allocated and evaluated; activity-front optimization is a later measured improvement.

7. Ruleset refactor

RuleSet currently owns a Canvas pointer because it also owns the dense sweep. Remove that pointer and make the transition callable by SparseCellGrid. Palette evaluation remains in the rule because visual colors are rule-specific.

```
class RuleSet {
public:
    virtual ~RuleSet() = default;

    virtual unsigned char evaluateNext(
        unsigned char currentState,
        unsigned char countedNeighbors) const = 0;

    virtual void evaluateColor(
        unsigned char state,
        unsigned char destinationRgb[3]) const = 0;

    virtual std::string getRuleTag() const = 0;

    bool supportsSparseBackground() const
    {
        return evaluateNext(1, 0) == 1;
    }
};
```

Migration rules

- Remove Canvas parameters from every concrete ruleset constructor.
- Rename protected nextState to public evaluateNext, or expose a non-virtual public wrapper around the protected override.
- Remove countAliveNeighbors and calcGeneration from RuleSet; neighborhood mechanics belong only to SparseCellGrid.
- Continue counting only state 0. In Wireworld that remains the electron head; in binary rules it remains alive.
- Validate evaluateNext(1,0)==1 when a rule is created or activated. Reject a future non-quiescent rule with a clear error.
- Keep Rule 90 and Rule 184 stubs as identity transitions until their separate semantics are implemented.

Ruleset-switch behavior

Preserve the current behavior of retaining raw cell bytes when the active ruleset changes. CanvasView rebuilds the visible palette targets. Unknown multi-state values under a binary rule are treated as non-alive by the current transitions and normalize on the next step.

8. Visible-region CanvasView

The simulation may contain millions of distant cells, but presentation should scale with the window. CanvasView samples only the padded visible cell rectangle into a bounded RGB texture and draws it once.

State owned by CanvasView

- Current visible CellRect with signed 64-bit origin and int width/height.
- targetRgb and displayRgb arrays sized to the current visible cell count only.
- RGB byte staging buffer and dirty AABB in visible-local coordinates.
- Last observed grid revision and palette revision.
- Texture capacity, actual view size, unit-quad mesh, shader handle, and texture handle.
- Non-owning pointers to SparseCellGrid, Camera, IRenderWindow, and Renderer.

Visible rectangle changes

```
1. Ask Camera for the four screen corners converted to cells.
2. Take the min/max cell coordinates and add one padding cell.
3. Allocate new CPU buffers for the new rectangle.
4. Fill targets from the grid by visible chunk, not one hash lookup per cell.
5. Copy display RGB for coordinates in the old/new overlap.
6. Initialize newly exposed display RGB directly to its target color.
7. Replace CPU buffers and mark the actual rectangle for upload.
```

Grid or palette changes

When the visible rectangle is unchanged but the grid or palette revision changes, fill background targets first, visit only chunks intersecting the visible rectangle, write their state colors, and compare against existing targets. Start fades and dirty only cells whose targets changed.

Texture capacity

- Allocate a reusable RGB texture whose capacity is at least the actual visible width and height.
- Grow each dimension geometrically, such as the next power of two, and never shrink during the session.
- Add explicit replaceTexture semantics to Renderer and IBackend so replacing an existing handle frees the old OpenGL object and is recorded by MockBackend.
- Upload the actual view into texture origin (0,0), set source row stride to the CPU view width, and pass uUvScale = actual/capacity.
- Keep staging buffers alive and unchanged until SubmitCommandQueue returns.

One-quad shader contract

```
uniform mat4 uMVP;
uniform vec2 uWorldOriginRelative;
uniform vec2 uWorldSize;
uniform vec2 uUvScale;
uniform sampler2D uDisplayTexture;

worldPosition = uWorldOriginRelative + unitPosition * uWorldSize;
TexCoord = unitUv * uUvScale;
gl_Position = uMVP * vec4(worldPosition, 0.0, 1.0);
```

Why not render one texture per chunk?
Per-chunk textures make GPU lifetime proportional to world history and multiply frame commands. A single visible texture keeps the token count constant and avoids turning CommandQueue's current 2048-token capacity into a hidden world-size limit.

9. Editor, startup, and controls

Painting

- Replace ScreenToWorld plus width/height checks with Camera::screenToCell.
- Keep iterative Bresenham stroke interpolation, but make all endpoints and loop coordinates signed 64-bit.
- Every interpolated cell calls SparseCellGrid::setCell; negative and distant coordinates are valid.
- Wireworld keeps left=conductor and right=empty until its separate head-placement UX is improved.
- If coordinate arithmetic or allocation fails, stop the stroke segment and log once; do not wrap.

Camera controls

- Middle-mouse drag stores previous screen coordinates and calls panByScreenPixels.
- Scroll calls zoomAtScreen with the cursor position; keep the existing 0.1 to 100 practical zoom clamp unless texture-capability validation requires a higher minimum.
- Reset centers the camera at cell (0,0), zero sub-cell offset, zoom 1, and rotation 0.
- Rotation remains supported by relative matrices even though the current CameraRotate path is empty.

Startup and clear

- Remove CanvasX/CanvasY allocation and center calculations.
- Seed the classic glider around the origin only when starting Game of Life.
- Start other rules, especially Wireworld, with an empty world.
- Clear erases the complete chunk map and immediately snaps the visible view to background, matching the destructive reset character of the current C action.

Obsolete configuration

- Stop creating, reading, and documenting CanvasX, CanvasY, and enableInfCanvas.
- Remove them from envvars.json defaults; tolerate stale keys in existing user files by ignoring them.
- Keep -cw and -ch parsing for one compatibility cycle, print that the option is ignored because the canvas is unbounded, and do not write environment variables.

10. Sparse save format and legacy import

New file layout

Write each scalar explicitly in little-endian order; never dump a C++ struct because padding and ABI layout are not a file format. The first release uses version 1 and fixed chunk size 16.

Field	Type / bytes	Meaning
magic	8 bytes	ASCII CSIMSP1 followed by NUL
version	uint32	1
chunkSize	uint32	16
flags	uint32	0; reserved
ruleTagLength	uint32	Number of following tag bytes; maximum 128
chunkCount	uint64	Number of chunk records
cameraAnchorX/Y	2 x int64	Exact saved camera anchor cells
cameraOffsetX/Y	2 x float64	Normalized sub-cell pixel offsets
zoom / rotation	2 x float64	Saved view state
ruleTag	variable	Known normalized ruleset name
records	repeated	int64 chunkX, int64 chunkY, then 256 state bytes

Deterministic writer

- Collect allocated chunk keys and sort by chunkY, then chunkX.
- Verify each chunk has a non-zero nonBackgroundCount and write all 256 state bytes.
- Write to a temporary file beside the requested destination.
- Flush and close successfully, then replace the destination atomically where the platform supports it.
- On failure, remove only the known temporary file and leave the previous destination untouched.

Transactional reader

- Read the first eight bytes. If they match CSIMSP1 NUL, parse the versioned format; otherwise rewind and parse the legacy format.
- Validate version, chunk size, flags, tag length, known rule tag, remaining file size, unique keys, and finite camera values before allocating the live world.
- Calculate the exact record bytes from file length before reserving chunkCount, preventing a forged count from causing a huge allocation.
- Reject any stored all-background chunk as non-canonical, or normalize it by skipping it; choose rejection for version 1 so corruption is visible.
- Parse into a temporary SparseCellGrid and temporary camera metadata. Publish nothing until EOF and all invariants pass.
- After validation, switch to the saved rule, swap the grid, restore the camera, rebuild the palette, and snap the visible view.

Legacy dense import

- Parse 128 rule-tag bytes, little-endian signed 32-bit width and height, and exactly width x height state bytes.
- Require positive dimensions, checked multiplication, a known rule tag, and sufficient file length.
- Map dense cell (x,y) to world cell (x,y); skip state 1 so empty space allocates nothing.

- Activate the saved rule automatically and center the camera on the occupied bounds. Use origin and zoom 1 for an empty legacy file.
- New saves always use the sparse format; do not write the legacy dense format.

11. Save/load integration

CellWorldIO should be testable without a window or file dialog. Platform SaveLoad continues to choose paths; CellGameModule coordinates the chosen path, live context, and camera.

Keyboard actions

- Add LeftControl and RightControl to KeyCode translation and InputManager's enumerated keys.
- Bind edge-triggered S and O actions. Execute only while either control key is down and the in-game console is closed.
- Ctrl+S requests a save path, then calls the same save method used by console commands.
- Ctrl+O requests a load path, validates into temporary state, and publishes only on success.
- Fix the Windows save panel to use overwrite prompting instead of requiring the destination file to exist.

Console actions without Game coupling

```
// Services/CommandLine exposes generic optional callbacks only.
using FileCommandHandler =
    std::function<bool(const std::string& path, std::string& message)>;

void setSaveHandler(FileCommandHandler handler);
void setLoadHandler(FileCommandHandler handler);
void clearFileHandlers();
```

- CellGameModule registers lambdas that invoke CellWorldIO after CellContext exists.
- CommandLine calls a handler only when save/load has a filename and reports the returned real result.
- Remove the current unconditional success message and commented-out legacy calls.
- CellGameModule clears handlers during Exit before deleting CellContext, preventing dangling captures.
- CommandLine includes no Game header and Illumo constructs no Game service.

12. Implementation sequence

The phases below are ordered to preserve a working reference path until the sparse domain is proven. Commit boundaries are suggestions; do not stage unrelated existing changes.

Phase	Goal	Main work	Exit gate
0	Protect baseline	Record status; build and run Release tests; note existing changes.	No source edits.
1	Coordinate primitives	Add CellAddress, ChunkAddress, hash, floor mapping, checked arithmetic.	New coordinate tests pass.
2	Sparse storage	Implement get/set/clear/pruning/revision without generation.	Storage tests pass; old Canvas still runs.
3	Rule API	Detach rules from Canvas and expose pure transition/palette contract.	Existing rule behavior tests pass through direct transitions.
4	Chunk stepping	Candidate set, 18 x 18 halo, next map, swap, failure rollback.	Dense-reference and boundary tests pass.
5	Context cutover	CellContext owns grid/rule/view; CellGameModule edits and steps grid.	Headless gameplay uses no lifeCanvas.
6	Relative camera	Anchor-cell camera, screen hit, pan/zoom, visible bounds.	Far-coordinate camera tests pass.
7	CanvasView	Visible buffers, fade overlap, texture capacity, one-quad tokens.	MockBackend view tests pass.
8	Persistence	CellWorldIO, new writer/reader, legacy parser, camera metadata.	Round-trip and malformed tests pass.
9	Product wiring	Ctrl actions, dialogs, console callbacks, startup/reset/config cleanup.	Windows manual smoke succeeds.
10	Documentation	Consensus, decision log, READMEs, AGENTS, benchmark instructions.	No dense/toroidal current-state claims remain.

Cutover discipline

- Do not begin CellGameModule cutover before the sparse engine matches a dense non-wrapping reference.
- Do not delete dense Canvas members until CanvasView tests cover palette, fade, dirty upload, and one draw.
- Do not replace save output until the new reader can round-trip and the legacy parser has fixtures.
- At each phase, build CSimTests explicitly before running the full default build.

13. File and symbol map

Area	Likely changes	Notes
Game domain	New SparseCellGrid.h/.cpp and CellCoordinates.h	Keep shared by app and tests; no rendering includes.
Canvas presentation	Replace Canvas internals or introduce CanvasView.h/.cpp	Prefer explicit CanvasView name; one texture and quad.
Rules	RuleSet and all concrete rule constructors/overrides	Pure transition API; no Canvas pointer.
Context/module	CellContext and CellGameModule	Own/wire grid, view, rules, editing, tick, startup, file actions.
Camera	Camera.h/.cpp	Anchor cell, sub-cell offset, screen-space operations, relative MVP.
Renderer	IBackend, Renderer, GLBackend, MockBackend, canvas shaders	Explicit texture replacement and visible-quad uniforms.
Persistence	New CellWorldIO plus current platform SaveLoad	Pure parser/writer; dialogs only choose paths.
Input/console	KeyCode, InputManager, CommandLine	Control keys and generic file callbacks.
Build/tests	CSim/CMakeLists and Tests	Put new shared sources in CSIM_SHARED_SOURCES.
Docs	architecture consensus, decision log, Game/root README, AGENTS	Supersede dense/toroidal facts and document commands.

Symbols removed or superseded

- Canvas::canvasWidth, canvasHeight, lifeCanvas, inBounds, and global-grid dirty rectangles.
- RuleSet::canvas, RuleSet::calcGeneration, and toroidal countAliveNeighbors.
- CellContext construction from CanvasX/CanvasY and getCellCanvas raw-domain usage.
- Camera reset based on CanvasX/CanvasY and absolute float world position.
- Dense SaveCellGame/LoadCellGame byte dumps.

Symbols retained conceptually

- Rule palette evaluation and cell encodings.
- CellGameModule tick accumulator, EDIT/NORMAL modes, and Scene contribution.
- RGB target/display fade behavior, bounded to the view.
- Renderer token path, PBO-backed UpdateTexture, and MockBackend test strategy.

14. Automated test matrix

Coordinate and storage tests

Scenario	Assertion
Cells -17,-16,-1,0,15,16,17	Each maps to the expected chunk and local index.
Write background to absent cell	No allocation and no revision increment.
Write first non-background cell	One chunk; count 1; revision increments once.
Overwrite with same state	No count or revision change.
Erase last non-background cell	Chunk removed; absent reads background.
Clear populated distant world	Zero chunks and one logical revision.
Extreme checked coordinates	No signed overflow or wrap.

Simulation tests

Scenario	Assertion
GoL blinker across x=15/16	Period-two result matches an interior blinker.
GoL glider across four chunks	Movement and population match dense reference.
Birth into absent neighbor chunk	Required output chunk is created.
Population dies completely	All output chunks are reclaimed.
Negative-coordinate oscillator	Same result as translated positive pattern.
No wrap	A left-edge pattern does not affect an unrelated distant coordinate.
Brian's Brain cross-boundary	Alive -> dying -> background states preserved.
Wireworld cross-boundary wire	Head/tail/conductor signal traverses chunk edge.
Every active rule, randomized seed	N sparse steps equal a padded non-wrapping dense reference.
Stable pattern	Revision does not change and view is not dirtied.

Reference test strategy

Keep a simple dense, non-wrapping reference implementation inside the test target only. Surround randomized finite patterns with enough background padding for the chosen generation count, run the same RuleSet transition, and compare every cell in the reachable region. Use fixed seeds and include translated copies that cross positive and negative chunk boundaries.

15. Rendering, persistence, and integration tests

CanvasView and MockBackend

- Visible rectangle at origin and at negative coordinates composes correct palette bytes.
- A stationary grid revision with no active fade emits no texture update.
- A visible cell change starts a fade and produces a bounded dirty upload.
- Moving the view copies overlapping display colors and snaps newly exposed cells to targets.
- Growing beyond texture capacity replaces the same handle and MockBackend records the replacement.
- Each canvas frame emits one DrawIndexed regardless of world chunk count.
- UV scale equals actual visible dimensions divided by capacity.
- Far-coordinate rendering uses small relative uniforms and finite matrices.

File tests

- Round-trip negative and widely separated chunks under every known ruleset.
- Two saves of the same world produce identical bytes.
- New files restore exact camera anchor, offsets, zoom, and rotation.
- Legacy fixture imports at (0,0), skips background cells, switches rule, and frames occupied bounds.
- Truncated header, unsupported version, wrong chunk size, unknown rule, duplicate key, all-background record, invalid float, and trailing/short record fail transactionally.
- Failed load leaves grid, rule, camera, palette revision, and view unchanged.
- Failed save leaves an existing destination unchanged.

Input and lifecycle

- Control key translation works for left and right controls.
- Ctrl+S/O are edge-triggered and ignored while the in-game console owns input.
- Console save/load reports handler success or failure rather than unconditional success.
- File handlers are absent before module Start and cleared before module Exit completes.
- A failed module Start still cannot call update or dispatch on an uninitialized CellContext; preserve or improve existing lifecycle guards while touching the module.

16. Benchmark and manual validation

Opt-in headless benchmark

Add a CSimChunkBench target that is not part of CTest pass/fail. Build Release and report median generation time, evaluated candidate chunks, cells evaluated, and chunks per second.

Dataset	Sizes	Purpose
Compact random chunks	1, 100, 1,000, 10,000	Steady dense-in-chunk throughput
Widely separated chunks	1, 100, 1,000, 10,000	Prove coordinate span does not create extra work
Large still life collection	Several scales	Observe no view revision and stable-map cost
Wireworld conductor field	Several scales	Expose static multi-state worst case
Expanding Life Without Death	Fixed seed, many generations	Measure growth and allocation behavior

```
cmake --build build --config Release --target CSimChunkBench
build\Release\CSimChunkBench.exe
```

Do not set a hardware-time threshold in CTest. Add structural assertions instead: candidate count is bounded by nine times source chunk count, and moving identical chunks far apart does not change evaluated cell count.

Windows OpenGL smoke test

- Launch at origin; confirm Game of Life seed and white unbounded background.
- Paint across x=15/16, x=-1/0, y=15/16, and y=-1/0 while paused.
- Run the simulation and watch patterns cross those boundaries without seams.
- Pan repeatedly into negative and distant space, paint, zoom, reset, and return to origin.
- Switch among all active rules; verify palette changes and Wireworld starts without glider heads.
- Clear a world containing distant chunks and confirm allocated count returns to zero.
- Save, modify, load, and confirm ruleset plus camera view restore.
- Load a known legacy dense file and confirm it appears near positive origin.
- Resize the window and exercise minimum/maximum zoom to force texture-capacity growth.

17. Failure modes and safeguards

Risk	Symptom	Required safeguard
Negative division	-1 maps into chunk 0	Single tested floor division/modulo implementation.
In-place generation	Order-dependent patterns	Old map read-only; publish with one map swap.
Missed candidate neighbor	Pattern stops at chunk edge	Expand every source chunk by all 8 neighbor keys.
Persistent halos	Border values drift or require sync	18 x 18 data exists only during one candidate evaluation.
Empty chunks retained	Memory grows after patterns die	Maintain exact non-background count and prune.
Per-cell hash reads	Zoomed-out view becomes CPU-heavy	Fill background once and copy intersecting chunks.
Per-chunk draw	Token overflow or hidden render limit	One visible texture and one canvas draw.
Far float coordinates	Jitter or wrong mouse cell	Anchor-cell camera and camera-relative GPU coordinates.
Partial load	World corrupts after bad file	Parse temporary grid/rule/view metadata and swap only on success.
Forged chunk count	Huge reserve or allocation failure	Validate record bytes against file length before reserve.
Dangling command callback	Crash after module exit	Clear handlers before deleting CellContext.
Non-quiescent rule	Infinite spontaneous birth	Require transition(background,0)==background.

No hard world cap

The chosen policy is to impose no fixed chunk count. That does not permit silent failure: checked coordinate math, transactional generation, transactional loading, accurate chunk counters, and clear allocation errors are mandatory. A later configurable budget may be added if real usage shows a need.

18. Documentation and decision updates

- Update architecture-consensus.md so current truth says sparse 16 x 16 chunks, temporary halo, non-wrapping generation, visible-region RGB fade, and one texture draw.
- Append a new decision entry, for example D-C2, that supersedes D-C1's combined Canvas and the dense/toroidal part of D-P3 without deleting history.
- Update the game/rules LaTeX chapter with storage complexity, quiescent-rule constraint, and save format; do not regenerate PDF auxiliaries unless explicitly requested.
- Update root and Game READMEs, controls, command-line option notes, test commands, and benchmark command.
- Update AGENTS.md because Canvas ownership, generation topology, canonical paths, environment variables, and build targets are durable repository facts.
- Remove statements that infinite chunks are deferred, while continuing to defer SYCL and parallel compute.

Suggested decision text

> **D-C2 - Sparse unbounded cell world**
> Split authoritative cell state from presentation. Store non-background multi-state bytes in sparse 16 x 16 chunks keyed by signed coordinates. Compute radius-one generations from temporary 18 x 18 read tiles without wrap, and render only the visible region through one bounded RGB fade texture. Supersedes D-C1 and the dense/toroidal mechanics of D-P3; preserves pure RuleSet transitions, token rendering, and serial deterministic v1.

19. Final acceptance checklist

- [ ] Existing worktree changes were recorded and preserved.
- [ ] Release CSimTests passes before and after the migration.
- [ ] Cell state is no longer stored in Canvas or a global dense rectangle.
- [ ] Missing cells read as state 1 and empty chunks never persist.
- [ ] Negative-coordinate mapping is unit tested at every boundary case.
- [ ] Every generation reads only the old map and swaps complete results.
- [ ] Cross-chunk equivalence passes for every active ruleset.
- [ ] The world has no toroidal wrap behavior.
- [ ] Camera picking, pan, zoom, and draw remain precise at distant cells.
- [ ] Visible fades use bounded memory; newly revealed cells snap.
- [ ] Canvas rendering remains one texture update and one draw at most.
- [ ] New saves are deterministic, sparse, versioned, and atomic.
- [ ] Legacy dense files import at origin and switch rules automatically.
- [ ] Invalid loads are transactional and cannot alter live state.
- [ ] Ctrl+S/O and console save/load report real outcomes.
- [ ] CanvasX, CanvasY, and enableInfCanvas are no longer runtime inputs.
- [ ] CSimChunkBench reports structural and timing data without gating CTest.
- [ ] Windows OpenGL smoke test covers editing, simulation, view movement, and files.
- [ ] Architecture, decision log, READMEs, LaTeX, and AGENTS describe the new truth.

Definition of done
A finite pattern may be edited and simulated at positive, negative, or distant signed coordinates with behavior independent of chunk seams and coordinate span. Domain memory scales with allocated non-background chunks; presentation memory scales with the visible window; saved state returns to the same ruleset and view.

Appendix A. Practical class skeleton

```
class SparseCellGrid {
public:
    using ChunkMap = std::unordered_map<
        ChunkAddress,
        CellChunk,
        ChunkAddressHash>;

    unsigned char getCell(const CellAddress& address) const;
    bool setCell(const CellAddress& address, unsigned char state);
    void clear();
    bool advance(const RuleSet& ruleSet);
    bool replaceWith(SparseCellGrid&& validatedGrid);

    std::uint64_t getRevision() const;
    std::size_t getAllocatedChunkCount() const;
    void getAllocatedChunkAddresses(std::vector<ChunkAddress>& out) const;
    const CellChunk* findChunk(const ChunkAddress& address) const;

private:
    static ChunkAddress chunkForCell(const CellAddress& address);
    static int localX(const CellAddress& address);
    static int localY(const CellAddress& address);
    static int localIndex(int x, int y);

    void buildCandidateSet(CandidateSet& out) const;
    void fillHalo(const ChunkAddress& address, Halo& out) const;
    ChunkMap chunks;
    std::uint64_t revision;
};
```

CanvasView update contract

```
void CanvasView::update(const RuleSet& rule, float dt)
{
    CellRect wanted = camera->getVisibleCellRect(1);
    if (wanted != visibleRect)
    {
        rebuildForVisibleRect(wanted, rule);
    }
    else if (grid->getRevision() != observedGridRevision
        || paletteRevision != observedPaletteRevision)
    {
        rebuildVisibleTargets(rule);
    }

    tickVisibleFade(dt);
    ensureTextureCapacity(visibleRect.width, visibleRect.height);
}
```

Publish order after a successful load

```
1. Validate the parsed ruleset tag is known and sparse-compatible.
2. Construct the new RuleSet without modifying CellContext.
3. Swap the validated temporary grid into CellContext.
4. Replace the active RuleSet.
5. Restore or derive the camera view.
6. Rebuild the CanvasView palette and snap visible buffers.
7. Log success only after all six operations complete.
```

Appendix B. Implementation notes

Keep the first version boring

- Use standard containers and explicit ownership; do not introduce a custom allocator into the chunk map until profiling proves a need.
- Use byte states, not bitboards, because current rules are multi-state and clarity is a project priority.
- Use one serial loop and one reusable halo buffer before investigating threads.
- Prefer clear helper names over compressed arithmetic; signed coordinate bugs are expensive to diagnose visually.
- Keep source files shared by app and CSimTests in CSIM_SHARED_SOURCES and preserve application-only Tracy behavior.
- Follow CONTRIBUTING: avoid auto, namespaces, recursion, and new third-party dependencies.

Useful implementation assertions

- Every retained chunk has nonBackgroundCount > 0.
- Every local index is in [0,255].
- Every visible texture update lies within capacity.
- Every generated candidate is within the representable chunk range.
- Every ruleset activated in the infinite world has a quiescent state-1 background.
- Every new-format save writes exactly chunkCount records and reaches EOF on validated load.

Recommended first coding session

```
1. Add CellCoordinates.h and tests for floorDivide16/floorModulo16.
2. Add CellChunk and SparseCellGrid get/set/clear only.
3. Add tests for allocation, pruning, negative coordinates, and revision.
4. Build CSimTests and stop there.

Do not touch Canvas rendering, Camera, save/load, or CellGameModule in that
first session. A narrow green foundation makes the later cutover tractable.
```

> **You have a reversible path**
> The existing dense Canvas remains a working visual reference through phases 1-4. Use it to compare palettes and visible behavior while the sparse domain is developed headlessly. Cut over only after the domain proves itself.